

Week 1 · Exercise 5 — EASY-CHATGPT: your first server (and your first real steer)

This is the start of the project you'll grow for the rest of the course. By the final week it will be an *agent* in a Docker container, built the “agents all the way down” way. Today it's the floor it stands on: **a working chatbot server you build yourself — by steering a coding agent, not by hand-typing it.**

You installed **OpenCode** in the pre-course setup. Point it at a capable model (see below) and put it to work. The point of this exercise is not the code — the agent can scaffold most of it in one go. The point is that **you decide what to build, you read what the agent wrote, and you make it run.**

Work **solo or in pairs** — your call. And the first on-site session is built for exactly this task: **Raimon is there to help you** when the agent, Docker, or your environment fights back. Come with it half-built and questions ready.

A word on the model you drive OpenCode with

A coding agent is only as good as the model behind it. A tiny local model **cannot** drive an agent through real software work — don't waste your afternoon trying. Use one of:

- **A capable endpoint your course provides**, if it has one;
- **your own cheap subscription** to an agentic-capable model (Qwen, DeepSeek, Pi, MiniMax; GPT-5.4-mini is almost there);
- and if you're serious about this craft, by June 2026 Claude Opus 4.8 and GPT-5.5 are the best agentic models you can rent.

Keep your local Ollama small model for the *experiments* in later exercises — not for driving the agent.

What to build — EASY-CHATGPT

A small but real chatbot server. **The requirements below are deliberately minimal — turning them into a working spec is part of the exercise.** Steer OpenCode to build:

- **A REST backend in FastAPI** that sits as a **proxy** between your chat client and the LLM: your frontend never calls the model directly — FastAPI receives the request, calls the LLM over the OpenAI-compatible API (pointing at your capable model *or* a local one by config only), and passes the answer back. Hold on to that shape; it is the one Week 2 reuses.
- **A frontend in vanilla JavaScript + HTML + CSS** — a chat window that **renders the model's replies as Markdown**, so code blocks, lists and headings come out formatted instead of raw. No framework, no React, no Svelte, no TypeScript.

- **A context view.** Show what's actually sent to the model and the token usage that comes back — the way the context-explorer demo from Lecture 3 did. You should be able to *see* the conversation growing and the tokens climbing.
- It must **run from docker compose up** — the user edits a `.env` file, brings it up, and it serves on a port of the form **666n** (pick one — 6661, 6662...).
- Config (`base_url`, `key`, `model`) lives in `.env` — never hardcoded, never committed (so add to `.gitignore`).

Build it in three versions — each a real step up:

- **Baseline — no streaming.** FastAPI calls the LLM, waits for the whole answer, and returns it; the chat window shows it when it arrives.
- **Advanced — streaming.** FastAPI streams from the LLM and **proxies that stream through to the frontend** (server-sent events), so the answer types out token by token instead of landing all at once. This is the proxy doing real work — relaying a live stream, not just forwarding a finished blob.
- **Best — vision.** If you point at a vision-capable model, let the user attach an image to a question and send it along with the query.

Optional extras you'll likely want later anyway: user accounts, chat history (SQLite or plain JSON).

You'll know it works when you can open the page in a browser, hold a multi-turn conversation with the model, and watch the context view grow turn by turn — and when a teammate (or Raimon) can clone your repo, edit `.env`, run `docker compose up`, and reach the same thing on the first try. That last part — *it runs on someone else's machine* — is the real bar.

How to work

1. Open OpenCode in a fresh folder, pointed at your capable model.
2. Tell it what you want — start small, iterate. **Read every file it writes.** When it does something you don't understand, ask it to explain, or fix it yourself. - when you ask something of a coding agent, you need to have a strategy to eval the quality of the outcomes.
3. Get it running locally first, then get `docker compose up` working. Your agent will help you. Be sure to ask your agent, or chatgpt or claude the meaning of any word you are unfamiliar. Use the experience to learn. For example : “what is a container” “what is the difference of a container and a virtual machine”, “how do I map a port on the container to a port on the host machine”, what is the difference between 127.0.0.1 and 0.0.0.0 ? ... are good questions to start if you already don't know the answers to. You may also create a markdown wiki with all the concepts you learn, ask your agent to build it for you. You can use obsidian to navigate that wiki.
4. Commit as you go (`git add / commit / push`) — the agent will offer to commit; you decide what.

You're learning to **steer, read, and verify** your coding agent — which is the whole job from here on.

What to deliver

- The **working project in your course repo** under `week1/exercise5/` (or your repo's agreed layout) — including the `docker-compose.yml` and a README that says how to run it.
- A **PDF** to Moodle (**W1-Ex5**) — **each student submits their own PDF** (both PDFs in a pair may have the same content, but each must include at the top: your full name, your partner's full name, and the link to your shared repository, as they appear on the Artemis platform). In your own words:
 - a screenshot of EASY-CHATGPT running (the chat + the context view);
 - which model you drove OpenCode with, and why;
 - one thing the agent got wrong that you caught and fixed;
 - one sentence on what you'd want it to do next (you'll find out in Week 2).